

Compiladores

Juliana da Costa Feitosa
juliana.feitosa@unesp.br
Docente UNESP - Bauru-SP

Plano de Ensino

1. Introdução à compilação
2. Analisador léxico
3. Analisador sintático descendente
4. Analisador sintático ascendente
5. Análise semântica
6. Sistema de execução básica de uma linguagem de programação
7. Organização do compilador

Bibliografia

1. **Compiladores : princípios, técnicas e ferramentas São Paulo : Pearson : Addison-Wesley 2008.**
2. SETZER, V. A Construção de um Computador: Campus, 1986.
3. AHO, A. ; SETHI, R. ULLMAN, J. D. Compiladores: Princípios, Técnicas e Ferramentas, Ed. LTC.,1995.
4. **KOWALTOWSKI, Tomasz. Implementação de linguagens de programação. Guanabara Dois, 1983.**

Provas e Trabalhos

- **P:** $(P1 + P2)/2$
- **T:** $(T1 + T2)/2$
- **MF:** Média Final

$$(P*0.6) + (T*0.4) = MF$$

Provas e Trabalhos

- Prova Substitutiva
 - Apenas para quem perder **P1** ou **P2**
- **E**: Exame
 - Tem que ter presença **acima de 70%**

$$\text{Nota Final} = (E + MF)/2$$

Aula 1

Introdução à compilação

Linguagens

- **Linguagem:** um conjunto de regras (métodos) para combinar um conjunto de elementos (símbolos)
- **Linguagem natural:** como as pessoas se comunicam
- **Linguagem de programação:** surgiu para superar a barreira de comunicação entre o homem e a máquina
- **Linguagem de máquina:** conjunto de instruções básicas que um processador pode executar

Linguagens

- **Linguagem de alto nível:** comandos parecidos à linguagem natural: PHP, Java, C#
- **Linguagem de baixo nível (ou linguagem de montagem):** quando os comandos são semelhantes à linguagem de máquina: Assembly

“Hello World” em Assembly

```
    global start
start:
%elifidn __OUTPUT_FORMAT__, elf64
    %ifdef UNIX                ; Solaris/OI/FreeBSD/NetBSD/OpenBSD/DragonFly
        %define SYS_exit      1
        %define SYS_write     4
    %else                      ; Linux
        %define SYS_exit     60
        %define SYS_write    1
    %endif

    global _start
_start:
%else
    %error "Unsupported platform"
%endif

    mov     rax,SYS_write
    mov     rdi,1              ; stdout
```

“Hello World” em Assembly

```
mov     rsi,msg
mov     rdx,len
syscall
mov     rax,SYS_exit
xor     rdi,rdi      ; exit code 0
syscall
```

section .data

```
msg     db      "Hello, world!",10
len     equ     $-msg
```

“Hello World” em Assembly

```
%ifdef NetBSD
section .note.netbsd.ident
    dd      7,4,1
    db      "NetBSD",0,0
    dd      2000000000 ; amd64 supported since 2.0
%endif
%ifdef OpenBSD
section .note.openbsd.ident
    align   2
    dd      8,4,1
    db      "OpenBSD",0
    dd      0
    align   2
%endif
section .text
%ifidn __OUTPUT_FORMAT__, macho64 ; MacOS X
    %define SYS_exit      0x20000001
    %define SYS_write     0x20000004
```

“Hello World” em JavaScript

```
console.log('Hello World!');
```

“Hello World” em Python

```
print('Hello World!')
```

Tradutor

- Tem como função traduzir uma linguagem para outra
- Tradutores de **linguagem de alto nível:**
 - **Compiladores**
 - **Interpretadores**
- Tradutores de **linguagem de baixo nível:**
 - **Montadores**

História

- Na década de 40, os computadores ainda eram programados em **Assembly**. Os computadores também tinham pouca memória
- Computador de programa armazenado (Von Neumann)
- Surgiu a necessidade de uma **linguagem de alto nível**

História

- Em 1954 IBM desenvolveu uma máquina chamada 704, primeira máquina de sucesso a ser comercializada.
- Os clientes perceberam que os custos de software eram maiores que o custo de hardware (que não era baixo).
- Muitos começaram a pensar em como **escrever código de forma mais eficiente**.

História

- Em 1953 John W. Backus desenvolveu **Speedcoding** para o IBM 701.
- Ele era um interpretador que tinha o foco em facilidade de uso em troca de recursos do sistema. Assim, era rápido desenvolver código em Speedcoding, porém era 10 a 20x mais lento que código escrito a mão.

História

- O interpretador de Speedcoding consumia muita memória. Por isso, não se tornou popular, mas Backus achava que o projeto era interessante. Ele achou que o problema do Speedcoding era que as fórmulas executadas eram interpretadas. Por isso, ele achou que seria melhor se as fórmulas fossem **traduzidas para código de máquina**.

História

- Com isso, surgiu **FORTRAN (FORmulas TRANslated)**, um projeto bem sucedido que já no ano de 1958, 50% dos programas eram escritos em FORTRAN, melhorando a produtividade.
 - <https://share.google/Mlr6m7mY2YYDz7ZXV>

Compiladores

- O nome compilador, criado nos anos 50, faz referência ao processo de composição de um programa pela reunião de várias rotinas de biblioteca
- O processo de tradução, considerado hoje a função central de um compilador, era então conhecido como programação automática

Compiladores

- Os primeiros compiladores (da linguagem FORTRAN) foram construídos quase em meados de 1957, quando técnicas de projeto e implementação de linguagens ainda não estavam muito desenvolvidas
- O projeto de um compilador se inicia pela escolha de uma gramática para a linguagem fonte

Compiladores

- Converter um programa de linguagem de alto nível em um programa em linguagem de montagem, o qual pode ser executado diretamente
- É um programa que lê um programa escrito na **linguagem fonte** e o traduz num programa equivalente, a **linguagem objeto**

Compiladores

- Recebe a primeira instrução do programa fonte, confere-a para ver se está escrita corretamente, converte-a para linguagem de montagem em caso afirmativo e passa para a próxima instrução, repetindo o processo até a última instrução.

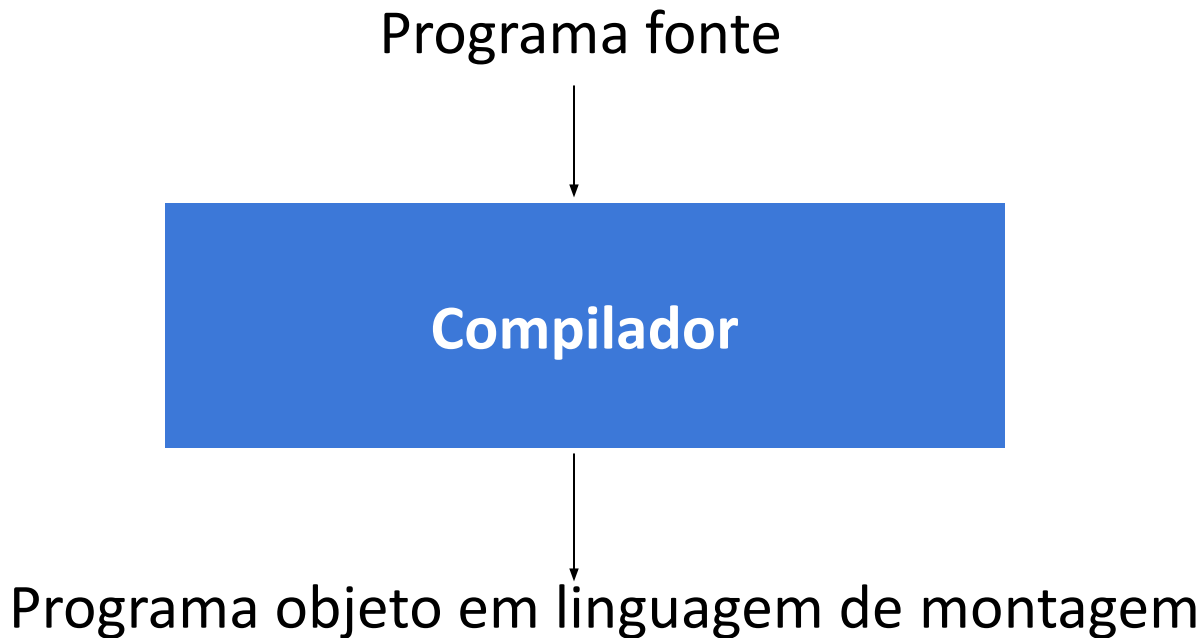
Compiladores

- Caso tenha terminado as transformações e nenhum erro tenha sido detectado (**o compilador relata ao usuário a presença de erros**), o computador volta à primeira instrução, já transformada para linguagem alvo (linguagem de montagem) e executa-a (em linguagem de máquina). Passa à instrução seguinte, executa-a e assim por diante.

Compiladores

- O Compilador necessariamente produz um arquivo objeto, que normalmente depois de serem feitas as ligações se transforma num **arquivo executável**, e sua execução ocorre em tempo de execução.

Compiladores



Compiladores



Benefícios do compilador

- “Embora compiladores não sejam explicitamente necessários para produzir código utilizável, a grande variedade e complexidade tanto das linguagens de programação quanto dos ambientes de máquina tornam os compiladores uma necessidade prática para criar software executável.”
 - <https://www.ibm.com/br-pt/think/topics/compiler>

Benefícios do compilador

- **Facilitar a programação em linguagens de alto nível**
 - Linguagens de programação de alto nível usam sintaxe e palavras-chave mais próximas das linguagens faladas, o que as torna muito mais fáceis de usar para desenvolvedores. Compiladores convertem esse código legível por humanos no código de máquina para executar aplicações de software otimizadas.

Benefícios do compilador

- **Reduzir repetição**
 - Ajudam a melhorar a eficiência convertendo código de alto nível em código de máquina executável. A saída do compilador é armazenada com a extensão de arquivo .exe, que é executada diretamente por um computador. Essa é uma tarefa realizada uma única vez.

Benefícios do compilador

- **Melhorar portabilidade**
 - Nem todos os sistemas conseguem executar todos os tipos de código de programação. Compiladores são usados para converter os tipos de código que desenvolvedores preferem usar nos tipos de código de que os sistemas precisam para operar.

Benefícios do compilador

- **Promover otimização geral**
 - Compiladores podem identificar e corrigir erros e falhas de software, resultando em programas mais estáveis e melhor otimizados. Também podem ajudar a melhorar a segurança do software, prevenindo erros relacionados a memória, como estouro de buffer, e gerando avisos se potenciais problemas de memória forem detectados.

Interpretadores

- Programa dentro de um sistema operacional que reconhece um conjunto de comandos do sistema e controla adequadamente o processador, tela e dispositivos de armazenamento.

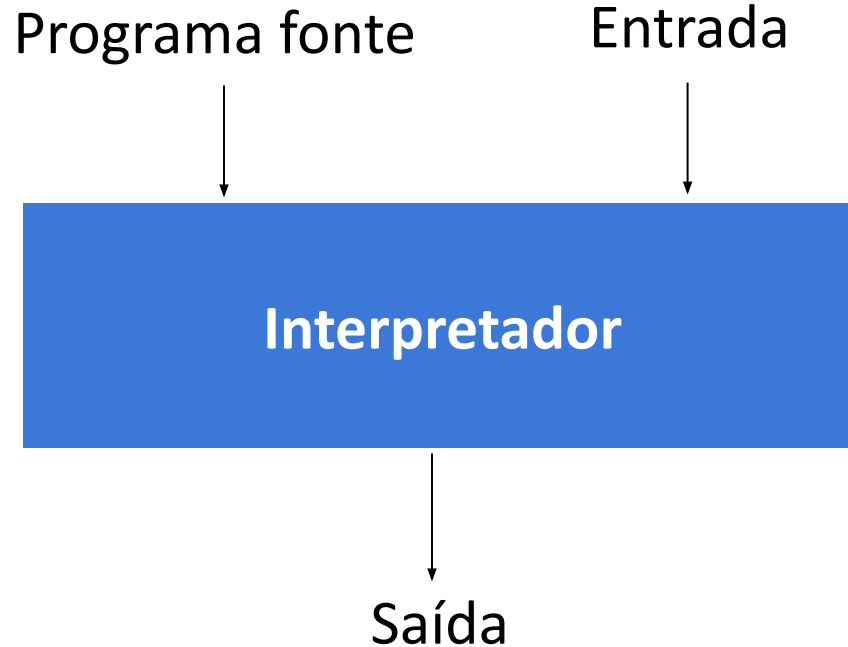
Interpretadores

- Em lugar de produzir um programa alvo (executável) como resultado de tradução, um interpretador realiza as operações especificadas pelo programa fonte.

Interpretadores

- Recebe a primeira instrução do programa fonte, confere para ver se está escrita corretamente, converte-a em linguagem alvo e então ordena ao computador que execute esta instrução. Depois repete o processo para a segunda instrução, e assim sucessivamente, até a última instrução do programa fonte.

Interpretadores



Interpretadores X Compiladores

- **Arquivo executável (Programa objeto):** Permite que o **compilador seja mais rápido**
- **Erros do programa:** Interpretador possui um melhor diagnóstico de erro

Montadores

- É o programa do sistema responsável por traduzir um programa *assembly* para o código de máquina. Esse processo de tradução de um programa fonte *assembly* para um programa em código de máquina é denominado montagem

Montadores

- É uma versão mnemônica do código de máquina, na qual são usados nomes em lugar do código binário para as operações e fornecidos nomes aos endereços de memória

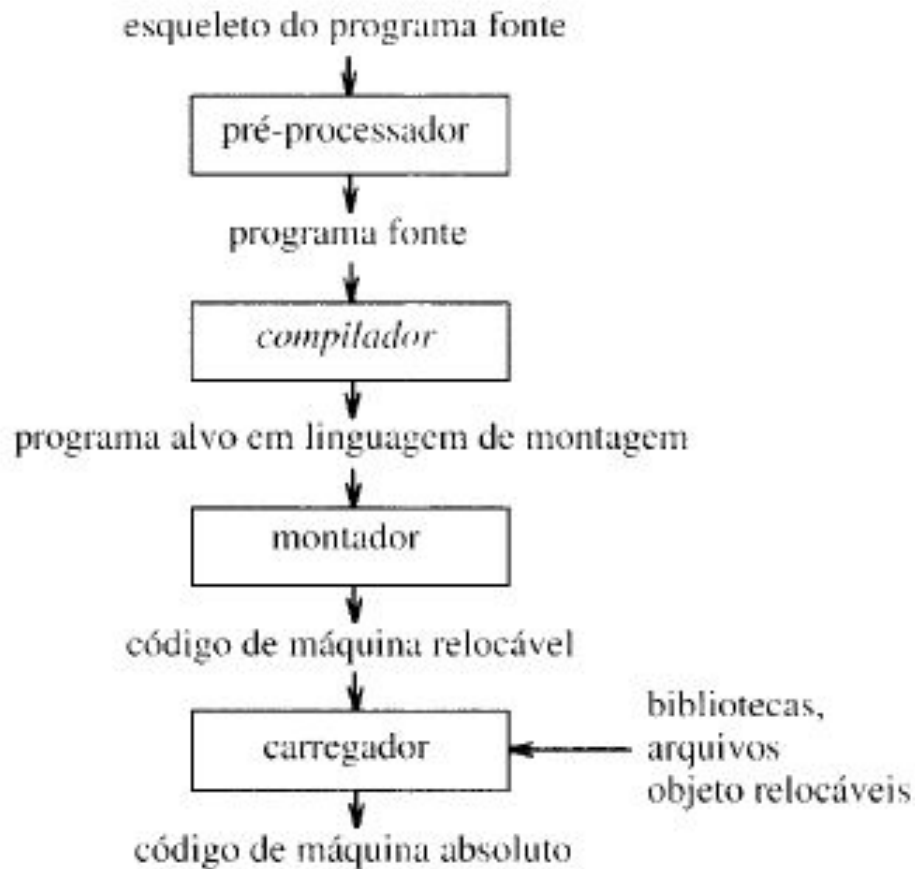
Montadores

Programa objeto em linguagem de montagem
(Assembly)

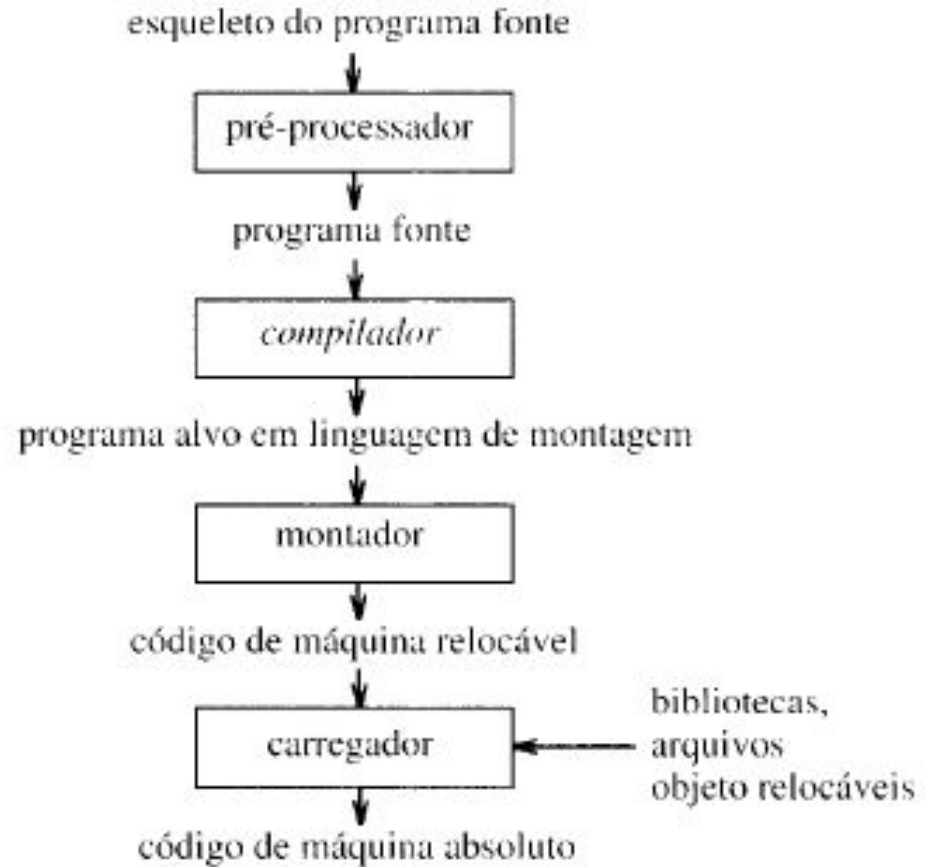


Código de máquina

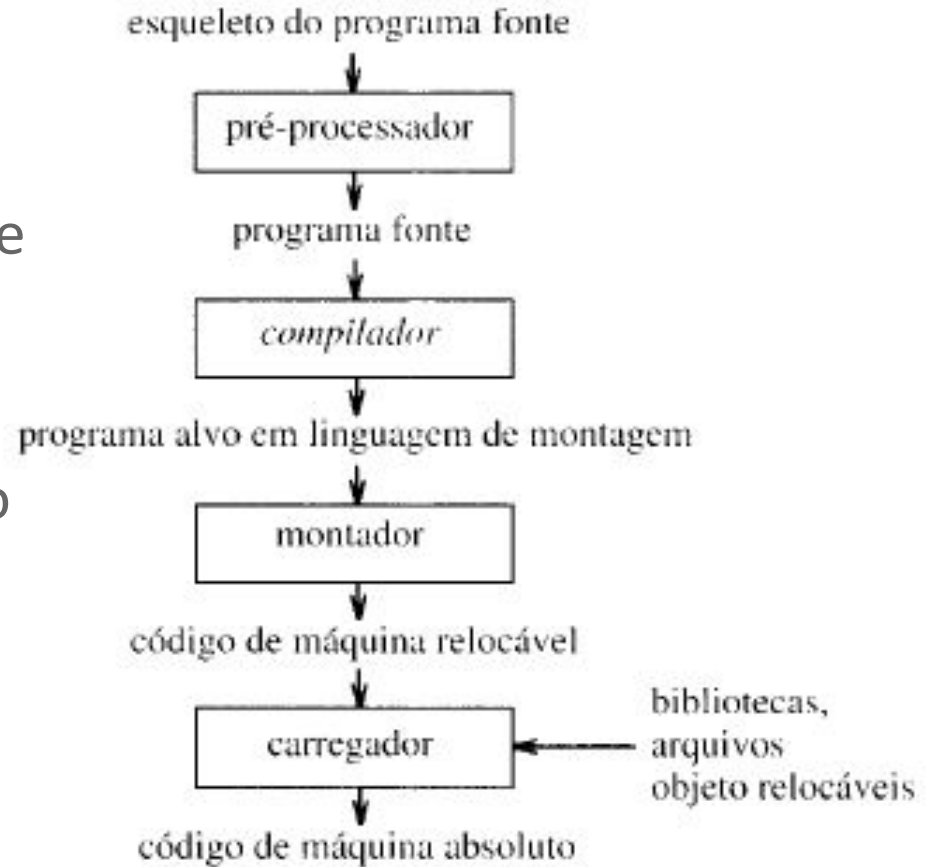
Sistema de processamento de linguagem



O programa objeto (alvo) criado pelo compilador pode exigir **processamento posterior** antes que possa ser executado



O compilador cria um **código de montagem** que é traduzido no de máquina por um montador, e então, ligado a algumas rotinas de biblioteca, formando o código que é efetivamente executado em máquina



Pré-processadores

- São responsáveis por produzir a entrada do compilador
- Podem realizar as seguintes funções:
 - **Processamento de macros**
 - **Inclusão de arquivos**
 - **Pré-processadores racionais**
 - **Extensores de linguagens**

Processamento de macros

- Um pré-processador pode permitir que o usuário crie abreviações para construções mais longas
 - **Exemplo:** `#define MAX 1000`
 - define **MAX** como abreviatura de **1000**. A sequência de caracteres que começa depois de MAX e vai até o fim da linha é o valor da macro (abreviatura)

Processamento de macros

- Exemplo: `#define MAX 1000`

O programa

```
#define MAX 1000

int main (void) {
    int v[MAX];
    for (int i = 0; i < MAX; ++i) {
        . . .
    }
    return EXIT_SUCCESS;
}
```

é transformado pelo pré-processador em

```
int main (void) {
    int v[1000];
    for (int i = 0; i < 1000; ++i) {
        . . .
    }
    return EXIT_SUCCESS;
}
```

Inclusão de arquivos

- Um pré-processador pode incluir arquivos no cabeçalho de um programa. Assim, se um programa possui um determinado enunciado, esse enunciado será substituído pelo arquivo referenciado
 - **Exemplo:** `#include <global.h>`
 - O pré-processador C faz com que o conteúdo do arquivo `<global.h>` substitua o enunciado `#include <global.h>`

Pré-processadores “racionais”

- Esse tipo de pré-processador expande as linguagens mais antigas com facilidades modernas de controle de fluxo e estruturação de dados
- Inclusão de rotinas não definidas na linguagem original
 - **Exemplo:** macros embutidas para comando como **while** e **if**, quando os mesmos não existem na linguagem de programação em si

Extensores de linguagens

- Confere maior poder às linguagens por meio de macros embutidas
 - **Exemplo: Equel (hoje SQL)** é uma linguagem de interrogação de banco embutida em C
 - Enunciados começados em **# #** são considerados pelo pré-processador como comandos de acesso ao banco de dados

Extensores de linguagens

- Exemplo: Eque1

```
main( )
{
  ## char EMPNAME[20];
  ## int SAL;
  while (READ(EMPNAME))
  {
    ##      RANGE OF X IS EMP
    ##      RETRIEVE (SAL = X.SALARY)
    ##      WHERE X.NAME = EMPNAME
    ##      {
    ##        PRINT("The salary of", EMPNAME, "is", SAL);
    ##      }
  }
}
```

Carregador (*loaders*)

- Normalmente é de responsabilidade do sistema operacional carregar e executar arquivos. A parte responsável por isso chama-se **carregador**
- Realiza duas funções: **de carga** e de **edição de ligações**

Carregador (*loaders*)

- **Processo de carga**
 - Alterar os endereços do código relocável de máquina para uma localização apropriada
- **Edição de ligações**
 - Permite criar um único programa a partir de diversos arquivos de código relocável de máquina (resolve as referências externas, como bibliotecas, arquivos, etc)